

Relazione del progetto individuale di Metodologie di Programmazione

Introduzione

Il progetto riguarda la riproduzione di una tipica battaglia Pokémon, nello specifico di una della prima generazione, con riferimento agli stessi meccanismi e elementi grafici. Tuttavia, essendo questa una riproduzione delle sole battaglie e non dell'intero mondo del gioco Pokémon, ho inserito le battaglie in un contesto arcade nel quale si può combattere contro avversari sempre più forti fino a che uno dei pokémon del giocatore, ossia l'utente che usa il programma, termina i suoi punti salute oppure raggiunge, secondo una certa condizione, la schermata di vittoria.

Classe JPokeBattle

La prima parte del programma è la classe main JPokeBattle che però si compone di poche righe di codice. Questo perché la sua funzione consiste solamente nel chiamare la classe Selezione (descritta nella sezione di seguito) che darà avvio alla prima finestra di gioco.

Classe Selezione

Come detto, la classe Selezione dà avvio alla prima finestra di gioco, necessaria alla scelta del Pokémon con cui iniziare il combattimento, che propone i tre classici starters, e un'area di testo mostrante le migliori dieci run compiute dal giocatore dall'avvio del programma; questo significa che una volta chiuso il programma la leaderboard avrà un reset dei suoi record.

La classe comprende pochi ma essenziali metodi.

Il primo è addWin() e permette semplicemente di tenere il conto delle vittorie consecutive avute in una run. Il secondo è passRecord() e lavora con il suo gemello nella classe Battaglia (che verrà descritta nelle prossime sezioni) per far rimbalzare tra di loro i dati necessari per un corretto accumulo dei record di tutte le run compiute; passaggio necessario poiché la classe Selezione fa decadere la sua finestra nel momento in cui passa la parola alla classe Battaglia per poi essere riaperta da quest'ultima più in là nel codice, lasciando ai due gemelli passRecord() il compito di mantenere i dati aggiornati e attivi in entrambe le classi, nonostante le loro frequenti chiusure e riaperture. Infine, il metodo endRun() va ad attingere dai dati di passRecord(), addWin() e i suoi parametri per aggiornare l'area di testo della leaderboard con l'ultima run eseguita, sempre che faccia parte delle migliori dieci. Una volta aperta la finestra della classe Battaglia, la finestra della classe Selezione viene chiusa.

Classe Pokemon

La classe Pokemon è semplice quanto essenziale. Ha il compito di "stampare" nuovi pokémon, con le relative statistiche e relative modifiche nel corso del gameplay. La classe è infatti composta essenzialmente da una lunga serie di metodi che vanno a restituire o impostare le informazioni essenziali del pokémon, ad esempio getPs() e setPs() hanno questo compito riguardo i punti salute del pokémon che vengono restituiti e modificati grazie a questi due metodi.

Compiti diversi hanno metodi come `damage()` o `update()` i quali, rispettivamente, restituiscono il danno inferto dal pokémon e modificano in blocco, con i parametri inseriti, le statistiche del pokémon; questi metodi hanno scopo importante per alcune scelte di programma che saranno spiegate nella classe Battaglia, descritta qui di seguito.

Nello specifico il danno inferto viene calcolato con l'uso di una leggermente modificata equazione canonica del gioco:

$$\text{Danno} = \{ \{ (2 * \text{Livello} / 5 + 2) * \text{Attacco} * 45 / \text{Difesa} \} / 50 \} + 2 \} * \text{STAB} * \text{Numero Casuale} / 10$$

STAB rappresenta l'efficacia della mossa contro l'avversario.

Assume valore 1 se i tipi sono uguali o non correlati (mossa normale → tipo normale / mossa fuoco → tipo normale)

Assume valore 2 se il tipo di mossa ha un vantaggio sul tipo di Pokémon.

Assume valore 3 se il tipo di mossa ha un vantaggio sul tipo di Pokémon ed è una mossa "speciale".

Maggiori dettagli nella sezione successiva.

Classe Battaglia

Questa è la classe centrale del programma che apre le finestre di combattimento in cui si svolge il gameplay vero e proprio. La finestra che viene aperta è quella con più elementi tra tutte le altre e segue un BorderLayout (come anche la finestra della classe Selezione) che divide gli elementi in tre sezioni, ossia in tre JPanel: superiore (NORTH), centrale (CENTRAL) e inferiore (SOUTH).

La parte superiore rappresenta le informazioni base della battaglia: gli stati dei due pokémon, quello del giocatore e quello avversario, con rispettivo livello del pokémon e punti salute rimanenti, tramite l'uso di JLabel; la barra degli xp del pokémon del giocatore, che restituisce un riferimento visivo alla progressione del livello del pokémon ogni volta che vince una battaglia; e, infine, un tasto di cura, ossia un piccolo extra che ho voluto inserire per rendere più semplice e divertente l'esperienza complessiva, dando l'opportunità al giocatore di curare di 35 punti salute il suo pokémon per un massimo di 5 volte per run.

La parte centrale è un'area di testo che funge da battle log, registrando le mosse eseguite dal giocatore e dagli avversari oltre che eventuali effetti di stato o informazioni utili. Altre informazioni come l'utilizzo della cura vengono invece segnalate attraverso finestre di dialogo (`JOptionPane.showMessageDialog()`).

La parte inferiore contiene tutte le operazioni essenziali che il giocatore può compiere durante il combattimento. La prima di esse è quella di poter cambiare pokémon; infatti, il giocatore, durante la sua run ha a disposizione una squadra composta da tutti e tre i pokémon starters, che possono essere usati nel combattimento ogni volta che si desidera tramite l'apposita tendina (comboBox) e tasto di selezione in qualsiasi momento. Ovviamente gli stati dei pokémon vengono mantenuti durante tutto l'arco della run e anche dopo esser stati cambiati con un altro pokémon, questo grazie al prima citato metodo `update()`. Il resto della parte inferiore contiene i tasti che eseguono le mosse del pokémon corrente; uno di essi all'inizio sarà vuoto poiché destinato a una nuova mossa che verrà appresa con l'acquisizione dei livelli.

Purtroppo altre mosse oltre le quattro destinate non saranno disponibili poiché ho progettato di far terminare il gioco prima che esse possano essere prese in considerazione; inoltre sin dall'inizio della run, come detto, 3 su 4 mosse saranno già conosciute essendo che il livello degli starters parte dal 5 (anche se tecnicamente la terza mossa dovrebbe essere appresa dal 6 o 7). Non è inoltre possibile che l'avversario attacchi prima del giocatore, ma sempre di seguito. Queste caratteristiche non canoniche sono scelte di programma prese per semplificare l'esperienza del gameplay che ho ritenuto troppo difficile, poiché per natura concedeva poche vittorie al giocatore per poi incorrere al game over.

Per lo stesso motivo il calcolo del danno ha subito delle variazioni rispetto al suo calcolo canonico: quando una mossa è superefficace lo è sempre per vantaggio del tipo della mossa sul tipo del pokémon (mossa tipo erba → pokémon tipo acqua) e va ad aggiungere un moltiplicatore all'equazione del danno; tuttavia, la quarta mossa, appresa in seguito, quando superefficace ottiene un moltiplicatore ancora più grande (x3 invece che x2), facendola diventare una sorta di mossa speciale. D'altro canto gli avversari non hanno possibilità di usare mosse di tipo diverso dal tipo normale, non avendo quindi occasione di avere colpi superefficaci, ritenuti troppo impattanti contro il giocatore; rimane invece possibile eseguire un colpo critico, sia per il giocatore che l'avversario, essendo che uno dei moltiplicatori dell'equazione del danno è un numero randomico.

Una volta che il pokémon avversario viene sconfitto viene scelto randomicamente un altro pokémon da quelli implementati e scelto come nuovo avversario. A questo scopo sono presenti delle HashMaps utili all'ottenimento, tramite il semplice nome del pokémon sorteggiato, delle statistiche base del pokémon stesso. In base poi a quanti nemici sono stati precedentemente battuti il livello del nuovo avversario viene incrementato e le sue statistiche di conseguenza, anche se non vengono aumentate tanto quanto quelle dei pokémon del giocatore nel momento di "level up" (sempre per un fattore di bilanciamento).

Quando uno dei pokémon del giocatore termina i suoi ps (game over) oppure riesce a raggiungere il livello 16, ossia quello necessario per l'evoluzione (you win), la finestra di battaglia viene chiusa. Nelle prime righe di codice della classe viene aperta una nuova finestra della classe Selezione in cui vengono restituiti i dati della leaderboard (passRecord()) a cui verranno poi aggiunti i dati della run appena conclusa con il metodo endRun(). Pertanto alla chiusura della finestra della battaglia rimarrà invece aperta la nuova e aggiornata finestra di selezione in cui poter iniziare una nuova run.

Classe CPU

Anche questa è una classe corta e semplice che ha il solo scopo di decidere l'azione che eseguirà l'avversario in base a poche e molto semplici condizioni per dare un minimo di variabilità al comportamento dell'avversario.

UML delle classi



